การทำงานกับ AI ในโปรเจกต์นี้

How AI Was Used

Version 1.0

ระบบจัดซื้อจัดจ้างขนาดย่อ (Mini e-Procurement)

จัดทำโดย บารเมษฐ์ นามมุงคุณ

ชื่อโครงการ	ระบบจัดซื้อจัดจ้างขนาดย่อ (Mini e-Procurement)
เลขที่สัญญา	ไม่มี (โจทย์สมมติเพื่อแสดงผลงาน)
ชื่อเอกสาร	การทำงานกับ AI ในโปรเจกต์นี้ (How AI Was Used)
วันที่เอกสาร	4 พฤศจิกายน 2569
Version เอกสาร	1.0

จัดทำโดย		
ชื่อ-นามสกุล	**ตำแหน่ง**	**วันที่**
บารเมษฐ์ นามมุงคุณ	System Analyst	4 พฤศจิกายน 2569

ตรวจสอบโดย		
ชื่อ-นามสกุล	**ตำแหน่ง**	**วันที่**
ยังไม่มี (เอกสารตัวอย่าง)	-	-

อนุมัติโดย		
ชื่อ-นามสกุล	**ตำแหน่ง**	**วันที่**
ยังไม่มี (เอกสารตัวอย่าง)	-	-

ประวัติการแก้ไข

Version	วันที่	ผู้แก้ไข	รายละเอียด
1.0	4 พฤศจิกายน 2569	บารเมษฐ์ นามมุงคุณ	จัดทำฉบับแรก (ร่างโดย AI รอผู้จัดทำตรวจรับ)

สารบัญ

1. หลักการ **1**
2. ข้อจำกัดของเอกสารที่ pipeline แก้ **1**
3. ขั้นตอนและบทบาท **2**
4. สิ่งที่ตัวตรวจจับได้และสิ่งที่ AI แก้ระหว่างทาง **4**
5. สิ่งที่ตรวจแล้วและสิ่งที่ยังไม่ได้ตรวจ **6**

สารบัญตาราง

ตารางที่ 2-1 ข้อจำกัดของเอกสารและวิธีที่ pipeline แก้ 2
ตารางที่ 3-1 ขั้นตอน บทบาทคนและ AI และการตรวจ 4
ตารางที่ 4-1 ปัญหาที่พบระหว่างทางและการแก้ 6
ตารางที่ 5-1 สิ่งที่ตรวจแล้วและสิ่งที่ยังไม่ได้ตรวจ 9

1. หลักการ

- **คนกำหนดโจทย์และขอบเขต AI ร่างและตรวจ:** ผู้จัดทำเลือกธุรกิจ ชุดเอกสารที่ต้องมี และข้อจำกัด ส่วนค่าสมมติทางธุรกิจ (เช่น เกณฑ์วงเงิน) AI เสนอและระบุเป็นสมมติไว้ในเอกสาร เพื่อให้ผู้จัดทำตรวจรับ
- **ให้สคริปต์ตรวจสิ่งที่ตรวจแบบแน่นอนได้:** ความสอดคล้องข้ามไฟล์ไม่ควรพึ่งการอ่านด้วยตาหรือความจำของ AI จึงเขียนตัวตรวจเป็นโค้ดและรันซ้ำได้ ตัวตรวจต้องถูกทดสอบเชิงลบด้วยว่าจับความผิดได้จริง
- **เอกสารเป็นไฟล์ย่อย เลขทุกตัวมาจากสคริปต์:** แต่ละข้อกำหนด กฎ เกณฑ์ตรวจรับ หน้าจอ และตารางข้อมูลเป็นไฟล์ย่อยหนึ่งไฟล์ในโฟลเดอร์ `design/` โดยไม่มีเลขข้อ เลขตาราง หรือเลขรูปอยู่ในไฟล์ สคริปต์ประกอบชิ้นส่วนเป็นเอกสาร ใส่เลขเอง และสร้าง PDF ไม่แก้ไฟล์ปลายทางด้วยมือ
- **เครื่องมือและสิ่งที่ไม่เผยแพร่:** เครื่องมือ AI ที่ใช้คือ Claude Code (Claude Sonnet 5.5) สำหรับร่าง แก้ข้ามไฟล์ รันสคริปต์ และดูภาพหน้าเอกสารที่สร้างจริงเพื่อหาข้อบกพร่องด้านการจัดหน้า; FigCraft (figcraft-cli v0.15.0) สำหรับสร้างและ render wireframe สคริปต์ที่ประกอบเอกสารและตรวจความสอดคล้อง รวมถึงไฟล์คำสั่งประจำโปรเจกต์ของ agent เก็บไว้ในเครื่องผู้จัดทำ ไม่ได้อยู่ใน repository นี้ repository จึงแสดงผลงานและหลักฐาน ไม่ใช่ชุดตั้งค่าที่ใช้รันซ้ำ

2. ข้อจำกัดของเอกสารที่ pipeline แก้

ประเด็น	ปัญหาที่พบเมื่อทำในโปรแกรมแก้ไขเอกสาร	วิธีที่ pipeline แก้
เลขข้อ	แทรกหรือย้ายหัวข้อแล้วเลขข้อและข้อความที่อ้างเลขคลาดเคลื่อน ต้องไล่แก้มือ	ไฟล์ย่อยไม่มีเลข สคริปต์กำหนดเลข 1, 1.1, 1.1.1 จากลำดับใน outline ทุกครั้งที่สร้าง และข้อความในไฟล์ไม่อ้างเลขข้อ (ผังที่เคยเขียนว่า "ไปผังที่ 2" ถูกแก้เป็นข้อความที่ไม่มีเลข)
เลขตารางและรูป	ตารางที่ X-Y ต้องใส่เอง หรือพิ่งฟิลด์ที่ต้องกดอัปเดตและเลขเปลี่ยนเมื่อมีตารางเพิ่ม	เลขมาจากเลขหัวข้อที่ตารางหรือรูปนั้นอยู่ ตามด้วยลำดับในหัวข้อ เช่น ตารางที่ 2.1-3 สร้างใหม่ทุกครั้ง
สารบัญ สารบัญรูปภาพ สารบัญตาราง	ต้องกด Update Field และเลขหน้าขึ้นกับโปรแกรมที่เปิดไฟล์	สร้างสามชุดอัตโนมัติ Chrome คำนวณเลขหน้าของรายการเองไม่ได้ จึง render หลายรอบ: รอบแรกหาหน้าที่หัวข้อและคำบรรยายไปตกจากข้อความใน PDF รอบถัดไปใส่เลขจนเลขไม่เปลี่ยน แล้วตรวจย้อนว่าทุกเลขตรงหน้าจริง
ตารางข้ามหน้า	หัวตารางไม่ซ้ำ แถวถูกตัดกลาง คอลัมน์รหัสแตกบรรทัด	หัวตารางซ้ำทุกหน้า แถวไม่ถูกตัด คอลัมน์ที่ค่าสั้นไม่ตัดบรรทัด และความกว้างคอลัมน์คำนวณจากความยาวเนื้อหา
การตัดคำไทย	ตัวแก้ไขเอกสารบางตัวตัดบรรทัดไทยได้เฉพาะที่ช่องว่าง คำจึงแตกกลางคำ	ใช้ Chrome ซึ่งตัดคำไทยด้วยพจนานุกรม
แผนภาพ	ภาพที่วาดแยกต้องวางและใส่คำบรรยายเอง	เขียนเป็นข้อความ Mermaid ในไฟล์ย่อย แสดงผลตอนสร้างเอกสาร และใส่เลขรูปอัตโนมัติ
ความสอดคล้องข้ามเอกสาร	รหัสที่อ้างกันระหว่างเอกสารตรวจด้วยตา	ตัวตรวจอ่านทุกไฟล์ย่อยและแผนงาน ตรวจว่ารหัสที่อ้างมีอยู่จริง ทุกข้อกำหนดมีเอกสารถัดไปรองรับ และไม่มีไฟล์ย่อยที่ไม่ถูกใช้ในเอกสารใด

ประเด็น	ปัญหาที่พบเมื่อทำในโปรแกรมแก้ไขเอกสาร	วิธีที่ pipeline แก้
ส่วนของเอกสาร header footer และเลขหน้า	หน้าแรกๆ (ปก ข้อมูลเอกสาร สารบัญ) ปนเลขหน้ากับเนื้อหา และ header/footer ของแต่ละส่วนต้องตั้งแยกในโปรแกรมแก้ไขเอกสาร	แบ่งเป็นส่วนเหมือน section ของไฟล์เอกสาร: ปก (รหัสเอกสารด้านบน จัดทำโดยด้านล่าง), หน้าข้อมูลเอกสาร, สารบัญ, สารบัญรูปภาพและสารบัญตาราง แยกหน้ากัน (ตัดสารบัญรูปภาพหรือสารบัญตารางออกเมื่อเล่มนั้นไม่มีรูปหรือตาราง) ไม่มีเลขหน้า; ส่วนเนื้อหามี header (ชื่อโครงการ ชื่อเล่มและรุ่น ผู้จัดทำ) footer (รหัสเอกสาร เลขหน้า) และเลขหน้าเริ่มที่ 1 หน้าแนวนอนใช้ header/footer ชุดเดียวกัน
สไตล์ตัวอักษร หัวข้อ และตาราง	รูปแบบตัวอักษรและหัวข้อของแต่ละเล่มต้องตั้งซ้ำและมักไม่เท่ากัน	ใช้ชุดสไตล์เดียวทุกเล่มตามแบบเอกสารราชการไทย: ฟอนต์ TH SarabunPSK 16 pt, หัวข้อบทตัวหนา 18 pt, หัวข้อรองไม่หนาและเยื้องตามระดับ, ย่อหน้าเยื้องบรรทัดแรกตามระดับหัวข้อ, ตารางเต็มความกว้างเส้นดำบาง หัวตารางสีเทาตัวหนากึ่งกลางและซ้ำทุกหน้า, คำบรรยายตารางและรูปตัวหนาเอียงกึ่งกลาง

ตารางที่ 2-1 ข้อจำกัดของเอกสารและวิธีที่ pipeline แก้

3. ขั้นตอนและบทบาท

ลำดับ	ขั้น	ผู้จัดทำกำหนด	AI ทำ	ผลลัพธ์	การตรวจ
1	โจทย์	ธุรกิจ (จัดซื้อจัดจ้างขนาดย่อ), ชุดเอกสาร SA ครบชุดแยกเป็นเล่มตามรายการเอกสารส่งมอบของโครงการภาครัฐทั่วไป (Timeline, User Requirement, User Flow, IA, System Architecture, Database, UX/UI, API, Test), ไม่ใช้ข้อมูลหน่วยงานจริง, เผยแพร่เฉพาะผลงานไม่เผยแพร่สคริปต์และไฟล์คำสั่งของ agent, โครงไฟล์ย่อยหนึ่งชิ้นหนึ่งไฟล์แบบ docs-as-code และต้องมีแผนการ์ดงานให้ dev	ถามตัวเลือกขอบเขตให้ผู้จัดทำเลือก อ่านโครงของ repository เอกสารอ้างอิงภายในของผู้จัดทำ (ไม่เผยแพร่) ทั้งไฟล์ย่อย แผนการ์ด และ pipeline และอ่านรายการเอกสารส่งมอบเพื่อแยกเล่ม	โครงโฟลเดอร์ `design/` และ `plan/` และ `design/outline.yaml`	ผู้จัดทำยืนยันก่อนเริ่ม
2	Requirements	– (เกณฑ์วงเงินและสายอนุมัติเป็นสมมติที่ AI เสนอ รอผู้จัดทำตรวจรับ)	ร่าง FR, BR, NFR, ผู้เกี่ยวข้อง, ขอบเขต และสมมติฐาน เป็นไฟล์ย่อยหนึ่งชิ้นหนึ่งไฟล์	`design/01-requirements/` (เล่ม User Requirement)	ตัวตรวจ: รหัสไม่ซ้ำ และทุกไฟล์ถูกใช้ในเอกสาร
3	กระบวนการและ Use Case	–	ร่าง Journey 4 รอบ ผังกระบวนการ สถานะ เอกสาร Use Case 10 รายการ และ sequence diagram	`design/02-bu-process/` (เล่ม User Flow)	ตัวตรวจ: ทุก Use Case อยู่ใน Journey เดียว และอ้าง FR ที่มีจริง
4	ข้อมูล	– (AI เลือก SQL Server T-SQL ให้ตรงกับเทคโนโลยีของตำแหน่งงานเป้าหมาย)	ออกแบบ 17 ตาราง constraint ดัชนี ขั้นตอนออกเลขเอกสารแบบไม่ซ้ำ และเหตุผลการออกแบบ	`sql/schema.sql`, `design/03-domain-data/` (เล่ม Database)	พจนานุกรมข้อมูลสร้างจากคอมเมนต์ใน SQL; รัน schema บน SQL Server 2022 จริง
5	API	–	เขียน OpenAPI 19 operation พร้อม `x-fr` และ `x-roles` รหัสข้อผิดพลาด 18 รหัส และกฎต่อ operation	`api/openapi.yaml`, `design/06-api-contract/` (เล่ม API)	ตัวตรวจ: `$ref` ทุกตัวหาเจอ ทุก operation มี FR และบทบาทที่ถูกต้อง

ลำดับ	ขั้น	ผู้จัดทำกำหนด	AI ทำ	ผลลัพธ์	การตรวจ
6	หน้าจอและ UX/UI	–	ร่างหน้าจอ 10 หน้า ฟิลด์และกติกา และกติกากลางของส่วนติดต่อผู้ใช้	`design/05-ui-desc/` (เล่ม UX/UI)	ตัวตรวจ: หน้าจออ้าง Use Case, FR และ operation ที่มีจริง
7	Wireframe ด้วย FigCraft	–	สร้าง wireframe ความละเอียดต่ำของทั้ง 10 หน้าจอด้วย FigCraft (figcraft-cli) จากสคริปต์เลย์เอาต์ที่ตรงกับฟิลด์และปุ่มในสเปกหน้าจอ ใส่เลขกำกับสีแดงอ้างอิงคำอธิบาย แล้ว render เป็น PNG ดูภาพจริงและแก้เลย์เอาต์จนอ่านได้	`design/05-ui-desc/wireframes/` (ไฟล์ต้นฉบับ `.figcraft`, ภาพ PNG และคำอธิบายเลขกำกับ ในเล่ม UX/UI)	ตัวตรวจ: ทุกหน้าจอมี wireframe เดียว ไฟล์ภาพและต้นฉบับมีจริง และการ์ดของหน้าจอนั้นอ้าง wireframe; ดูภาพทุกหน้าจอแล้วแก้ 4 จุด
8	โครงสร้างข้อมูลของส่วนติดต่อผู้ใช้ (IA)	–	เขียนหลักการ IA ศัพท์เฉพาะ และกติกากรณีขอบเขต แล้วให้สคริปต์สร้างเมทริกซ์สิทธิ์ของบทบาทกับหน้าจอและกับ API operation และเมทริกซ์ข้อมูลของแต่ละหน้าจอ	`design/05-ui-desc/ia/`, `design/05-ui-desc/glossary/` (เล่ม IA)	ตัวตรวจ: บทบาทของหน้าจอต้องไม่เกินที่ API operation ของหน้าจอนั้นอนุญาต
9	สถาปัตยกรรมและความมั่นคงปลอดภัย	– (เทคโนโลยีที่เลือกเป็นข้อเสนอของ AI รอผู้จัดทำตรวจรับ)	ร่างบริบท หน่วยประกอบ โมดูลของ API มุมมองขณะทำงาน การติดตั้ง ขอบเขตความเชื่อถือ มาตรการหลัก ความเสี่ยงตาม OWASP มาตรการต่อ NFR และ ADR 6 เรื่อง	`design/04-arch-desc/`, `design/08-security-arch/` (เล่ม System Architecture)	ตัวตรวจ: ทุก API operation อยู่ในโมดูลเดียว ทุก NFR มีมาตรการเชิงสถาปัตยกรรม หลักฐานของ OWASP ชี้เกณฑ์ตรวจรับที่มีจริง ทดสอบเชิงลบ 4 แบบ จับได้ทุกแบบ
10	เกณฑ์ตรวจรับ (Acceptance Criteria)	–	เขียนเกณฑ์ตรวจรับ (Acceptance Criteria) 52 ข้อ ผูกกับ FR ใช้การวิเคราะห์ค่าขอบกับเกณฑ์วงเงิน แล้วส่งต่อให้ QA ออกแบบ test case และข้อมูลทดสอบเอง	`design/01-requirements/acceptance-criteria/` (เล่ม Acceptance Criteria)	ตัวตรวจ: ทุก FR และทุก API operation มีเกณฑ์ตรวจรับ รหัสข้อผิดพลาดที่ระบุมีจริง และทุกข้อเป็นเกณฑ์ของการ์ดอย่างน้อยหนึ่งใบ
11	แผนการ์ดงาน	–	ตัดการ์ดหนึ่งใบต่อหน้าจอ (10 ใบ) และการ์ดสนับสนุน 3 ใบ จัด Journey เข้า Sprint 4 รอบ ประมาณการ story point ตั้งต้น	`plan/` (เล่ม Timeline and Implementation Plan)	ตัวตรวจ gate: ทุกหน้าจอและ API operation มีเจ้าของการ์ดเดียว Journey จบใน Sprint เดียว dependency ไม่ย้อนและไม่วนรอบ ทดสอบเชิงลบ 4 แบบ จับได้ทุกแบบ
12	เนื้อหาที่สร้างอัตโนมัติ	–	เขียนตัวสร้างพจนานุกรมข้อมูล ดัชนี endpoint เมทริกซ์การสืบกลับ เมทริกซ์ IA และตารางในการ์ด (Acceptance Criteria, Source Refs, Touchpoints)	`design/_generated/`, `plan/CARD-SLICING.md`, `plan/JOURNEY-SPRINT.md`, `plan/backlog.csv`	รันซ้ำแล้วไฟล์ต้องไม่เปลี่ยน (`--check`)

ลำดับ	ขั้น	ผู้จัดทำกำหนด	AI ทำ	ผลลัพธ์	การตรวจ
13	ประกอบเล่มและสร้าง PDF	–	เขียนตัวประกอบที่ใส่เลขข้อ เลขตารางและรูป สร้างสารบัญสามชุด และ render PDF 11 เล่ม ด้วย Chrome แล้วดูภาพหน้าที่ได้เพื่อหาข้อบกพร่อง	`output/*.pdf`	ตรวจเลขหน้าในสารบัญตรงทุกรายการ ไม่มีหน้าว่าง ดูภาพตัวอย่างหน้าที่เสี่ยง (แผนภาพ ตารางกว้าง)
14	ทดสอบฐานข้อมูล	–	เขียนชุดตรวจสอบ constraint สิทธิ์ audit และการออกเลข 1,000 เลขพร้อมกัน รันบน SQL Server 2022 ใน Docker พร้อมตัวควบคุม	`sql/tests.sql`, `tests/db/RESULTS.md`	ผ่านทั้งหมด ตัวควบคุม (`MAX()+1`) ได้เลขซ้ำจำนวนมาก

ตารางที่ 3-1 ขั้นตอน บทบาทคนและ AI และการตรวจ

4. สิ่งที่ตัวตรวจจับได้และสิ่งที่ AI แก้ระหว่างทาง

เรื่อง	เกิดอะไรขึ้น	แก้อย่างไร
ช่องว่างในเกณฑ์ตรวจรับ	ตัวตรวจรายงานว่า operation `getPurchaseRequest` ไม่มีเกณฑ์ตรวจรับเลย ทั้งที่ตาราง FR ดูครบ	เพิ่ม AC-02-03 และ AC-02-04
การออกแบบ endpoint อนุมัติ	ร่างแรกให้อนุมัติผลเลือกผู้ชนะผ่าน endpoint ของ PR โดยแยกด้วยฟิลด์ `target` ซึ่งปนความรับผิดชอบ	เปลี่ยนเป็น endpoint กลาง `/approvals` ที่แยกชนิดงานด้วย `docType` แล้วแก้ sequence diagram ให้ตรง
schema สร้างไม่ผ่านตอนรันจริง	รัน `sql/schema.sql` บน SQL Server ครั้งแรกแล้ว filtered index ล้มเหลว เพราะ `sqlcmd` ตั้ง `QUOTED_IDENTIFIER` เป็น OFF โดยปริยาย ต่างจากการเชื่อมต่อของแอป การอ่านด้วยตาไม่เจอ	เพิ่ม `SET ANSI_NULLS ON; SET QUOTED_IDENTIFIER ON;` ที่หัวไฟล์ แล้วรันซ้ำบนฐานข้อมูลเปล่าจนผ่าน
บั๊กในตัวทดสอบที่ AI เขียนเอง	AC-06-03 ล้มเหลวครั้งแรก เพราะอ้าง `quotation_id = 1` แต่ insert ที่ล้มเหลวก่อนหน้าใช้เลข identity ไปแล้ว	แก้ให้ UPDATE ตามคีย์ธุรกิจแทนเลข id schema ไม่ได้ผิด
ตรวจว่าการทดสอบจับความผิดได้	ผลผ่านอย่างเดียวพิสูจน์ไม่ได้ว่าตัวทดสอบมีประโยชน์	เพิ่มตัวควบคุม: วิธี `MAX()+1` แบบไม่ล็อกได้เลขซ้ำจำนวนมาก ขณะที่ `usp_next_doc_no` ได้ 1,000 เลขไม่ซ้ำ
แผนภาพทับกัน	แผนภาพสถานะไปวาดทับในผังกระบวนการและรูปของมันเองว่างเปล่า พบจากภาพที่ render เพราะ Mermaid สร้างรหัสองค์ประกอบจาก `Date.now()` ซึ่ง Chrome ที่รันแบบ virtual time คืนค่าซ้ำ รหัสจึงชนกัน	บังคับรหัสแบบเรียงลำดับและเรียก render ครั้งเดียว (การลองเรียกทีละภาพทำให้ตัวนับเริ่มใหม่ทุกครั้งและแย่ลงก่อนจะแก้ถูก)
ตัวตรวจแผนงานหลวมเกินไป	ทดสอบเชิงลบแล้ว Journey ที่ถูกแยกข้าม Sprint ได้ verdict `BLOCKED` แต่ตัวตรวจยังจบสำเร็จ และล้มด้วย traceback เมื่อการ์ดอ้าง operation ที่ไม่มีอยู่	ให้ `BLOCKED` นับเป็น error และให้ข้อความ error แทน traceback แล้วทดสอบซ้ำจนจับได้ทั้ง 4 แบบ

เรื่อง	เกิดอะไรขึ้น	แก้อย่างไร
ข้อบกพร่องการจัดหน้า	ผังกระบวนการยาวจนตัวอักษรเล็ก, คอลัมน์รหัสอย่าง `UC-01` แตกบรรทัด, ตารางข้อมูลทั่วไปของแต่ละหน้าจอทำให้สารบัญตารางยาว, และหน้าปกดันให้เกิดหน้าว่าง	แยกผังเป็นหลายภาพ, กำหนดไม่ตัดบรรทัดในคอลัมน์ค่าสั้น, เปลี่ยนเป็นรายการสั้นๆ ที่ไม่ใส่เลขตาราง, และลดความสูงของหน้าปก
เนื้อหาที่ขาดตอนแยกเล่ม	เมื่อแยกเล่มตามรายการเอกสารส่งมอบของโครงการภาครัฐ พบว่าชุดเดิมยังไม่มี IA และ System Architecture เลย และเล่มเดิมรวมหลายเรื่องไว้เล่มเดียว (ข้อกำหนดหน้าจอปนกับ Design System, กระบวนการปนกับ Use Case)	แยกเป็น 11 เล่มตามหัวข้อ เขียนไฟล์ย่อยใหม่ของ IA, สถาปัตยกรรม และความมั่นคงปลอดภัย และเพิ่มตัวตรวจของเนื้อหาใหม่ (บทบาทของหน้าจอ โมดูลของ API มาตรการต่อ NFR หลักฐาน OWASP) แล้วทดสอบเชิงลบ
ที่เก็บ Idempotency-Key ไม่มีใน schema	ตอนเขียนสถาปัตยกรรมพบช่องว่างในการออกแบบเดิม: ข้อกำหนด API ให้เก็บผลของ Idempotency-Key 24 ชั่วโมง แต่ schema ไม่มีที่เก็บ ตัวตรวจข้ามเอกสารไม่จับเพราะไม่มีรหัสใดอ้างถึง	ตัดสินใจใน ADR-04 ให้เก็บในแคช (Redis) พร้อม TTL และระบุความเสี่ยงว่าถ้าแคชหาย การส่งซ้ำอาจทำงานซ้ำ จึงต้องมีกฎสถานะของเอกสารเป็นด่านที่สอง เพิ่มเกณฑ์ตรวจรับ AC-03-07 สำหรับกรณีแคชหายแล้ว (ยังไม่ได้รันกับระบบจริง)
เลขหน้าเริ่มใหม่ที่เนื้อหาไม่ได้	ตั้งให้เลขหน้าเริ่มที่ 1 หลังหน้าสารบัญด้วย `counter-reset` ของ Chrome ลองทั้งแบบบน element และบน `@page` แล้วเลขยังนับต่อจากหน้าปก	สร้างหน้าหน้า (ไม่มีเลข) กับเนื้อหา (เลขเริ่ม 1 เอง) เป็น PDF สองไฟล์แล้วรวมด้วย `pdfunite` ผลพลอยได้คือเลขในสารบัญเป็นเลขหน้าเนื้อหาโดยตรง ตัดขั้นตอน render วนหลายรอบออกได้
พาธไฟล์ที่ค้างหลังย้ายไฟล์	ตอนแยกเอกสารเป็นเล่ม ข้อความสองจุดยังอ้างพาธ `docs/...` ที่ไม่มีแล้ว พบจากภาพหน้า PDF ไม่ใช่จากตัวตรวจ เพราะเป็นข้อความอิสระ	เพิ่มกฎในตัวตรวจ: พาธไฟล์ในเครื่องหมาย backtick ต้องมีอยู่จริง กฎนี้ตรวจเจอทั้งสองจุดและอีกสองจุดที่เขียนแค่ชื่อไฟล์ไม่ระบุโฟลเดอร์ แก้ข้อความให้ระบุพาธเต็ม
หน้าว่างท้ายเล่มที่มีแผนภาพ	สามเล่มมีหน้าว่างท้ายเล่ม 1 หน้า เพราะ Mermaid ทิ้งองค์ประกอบชั่วคราวไว้ท้ายหน้า ทำให้เกิดหน้าใหม่ที่ไม่มี header และ footer	ลบองค์ประกอบที่ไม่ใช่เนื้อหาหลัง render และเพิ่มการตรวจหน้าว่างทุกเล่ม
ข้อบกพร่องของ wireframe ที่เห็นจากภาพ	ดู wireframe ที่ render จริงแล้วพบข้อบกพร่องที่สคริปต์เลย์เอาต์ไม่รู้เอง: เลขกำกับสีแดงทับป้ายชื่อช่อง, ไฮไลต์ราคาต่ำสุดใน SCR-05 ไปอยู่แถวรายการแทนแถวราคารวม, ชิปสายอนุมัติใน SCR-02 ล้นกรอบ, เลขกำกับทับขอบปุ่ม	วางเลขกำกับในช่องว่างด้านซ้ายของเนื้อหา คำนวณตำแหน่งไฮไลต์จากแถวและคอลัมน์ของตาราง เรียงชิปสายอนุมัติเป็นแนวตั้ง แล้ว render ซ้ำจนอ่านออก
เมทริกซ์ไม่โยงไปถึงการ์ดงาน	การสืบกลับเป็นทางเดียว: การ์ดอ้างถึงเอกสารออกแบบได้ แต่เมทริกซ์ไม่มีคอลัมน์การ์ด จึงตอบไม่ได้ว่า FR ข้อหนึ่งอยู่การ์ดไหนและเสร็จหรือยัง และสถานะการ์ดมีค่าเดียวคือ draft ไม่มีคำจำกัดความที่ใช้ติดตามความคืบหน้า	เพิ่มคอลัมน์การ์ดและสถานะในเมทริกซ์ FR และ API operation เพิ่มตารางเกณฑ์ตรวจรับไปยังการ์ด กำหนดสถานะ 5 ค่า (draft, ready, in_progress, done, blocked) พร้อมผู้เปลี่ยน ตัวตรวจปฏิเสธค่านอกชุดนี้ และสรุปความคืบหน้าตาม Journey ด้วย story point ที่เสร็จ แก้สถานะในการ์ดแล้วสร้างใหม่ ตัวเลขทุกตารางเปลี่ยนตาม
ค่าอ้างอิงถูกพิมพ์ซ้ำด้วยมือ	ค่าที่มีต้นทางอยู่แล้วถูกพิมพ์ซ้ำด้วยมือในหลายไฟล์ (จำนวนตาราง จำนวน operation จำนวนข้อเกณฑ์ตรวจรับ ใน README ขั้นตอนงาน และการ์ด) และเมื่อแก้ต้นทางต้องไล่หาที่ที่ต้องแก้ตามต่อด้วยการอ่านเอง ซึ่งพลาดได้และสิ้นเปลืองโทเค็น	ค่าที่คำนวณจากต้นทางได้ใช้ตัวแทนในวงปีกกาคู่ (ชนิด count, title, field) ที่เครื่องมือเติมให้ตอนประกอบเล่มและตอนสร้าง README; ตัวตรวจ lint ปฏิเสธตัวเลขที่พิมพ์ซ้ำกับค่าที่คำนวณได้; เพิ่มคำสั่งวิเคราะห์ผลกระทบ (รหัสนี้ถูกอ้างจากที่ใด ทั้งทางตรงและทางอ้อม อยู่เล่มไหน) และคำสั่งเปลี่ยนชื่อรหัสทุกจุดพร้อมโหมดทดลอง AI ใช้คำสั่งเหล่านี้แทนการไล่อ่านและแก้เอกสารเอง แล้วทดสอบด้วยการเปลี่ยนชื่อ operation หนึ่งตัวและรันตัวตรวจ ซึ่งผ่านโดยไม่ต้องแก้มือ

เรื่อง	เกิดอะไรขึ้น	แก้อย่างไร
ความสัมพันธ์ระหว่างไฟล์ประกาศไม่ครบ	ความสัมพันธ์ระหว่างไฟล์ย่อยประกาศใน front matter ไม่ครบ: หลายประเภท (การตัดสินใจ ผังกระบวนการ สถานะ มุมมองสถาปัตยกรรม ดัชนี) ไม่มีฟิลด์ความสัมพันธ์เลย และหลายไฟล์พูดถึงรหัสอื่นแค่ในเนื้อความ ตัวตรวจจึงตรวจไม่ได้ว่าความสัมพันธ์นั้นมีอยู่จริงหรือครบหรือไม่	เพิ่มไฟล์กติกาความสัมพันธ์ต่อประเภทไฟล์ (ฟิลด์ไหนชี้ชนิดไหน บังคับหรือไม่ ประเภทไหนอยู่เดี่ยวได้) เป็นข้อมูล ไม่ใช่โค้ด และไม่ระบุว่ารหัสไหนผูกกับรหัสไหน ตัวตรวจกลางตรวจว่ารหัสในฟิลด์มีจริงและชนิดถูก ฟิลด์บังคับไม่ว่างประเภทไฟล์ใหม่ต้องมีกติกา และรหัสที่พูดถึงในเนื้อความต้องประกาศใน front matter ด้วย จากนั้นเติมความสัมพันธ์ที่ขาดใน 100 กว่าไฟล์ ทดสอบเชิงลบ 5 กรณี (รหัสไม่มี ชนิดผิด ฟิลด์บังคับว่าง พูดถึงโดยไม่ประกาศ โฟลเดอร์ใหม่ไม่มีกติกา) ตัวตรวจจับได้ทุกกรณี

ตารางที่ 4-1 ปัญหาที่พบระหว่างทางและการแก้

5. สิ่งที่ตรวจแล้วและสิ่งที่ยังไม่ได้ตรวจ

ตรวจแล้ว (มีหลักฐาน)	ยังไม่ได้ตรวจ
รหัสที่อ้างกันระหว่างไฟล์ย่อยและแผนงานมีอยู่จริงทั้งหมด (ตัวตรวจ 0 error)	เกณฑ์ตรวจรับ (AC) ยังไม่ได้ถูก QA ใช้ออกแบบ test case และยังไม่ได้รันกับระบบจริง เพราะยังไม่มีระบบ
ทุก FR มี Use Case, API และเกณฑ์ตรวจรับ; ทุก operation มีเกณฑ์ตรวจรับและมีเจ้าของการ์ดเดียว	ประสิทธิภาพ (NFR-01) ยังไม่ได้วัดบนข้อมูล 100,000 PR

ตรวจแล้ว (มีหลักฐาน)	ยังไม่ได้ตรวจ
`sql/schema.sql` รันผ่านบน SQL Server 2022 และชุดตรวจสอบ schema ผ่านทั้งหมด (`tests/db/RESULTS.md`)	การออกเลขทดสอบด้วย 20 การเชื่อมต่อบนเครื่องเดียวภายใต้การจำลอง amd64 ไม่ใช่ภาระจากเครือข่ายจริง
ตัวตรวจ gate ของแผนงานถูกทดสอบเชิงลบ 4 แบบและจับได้ทุกแบบ	ประมาณการ story point เป็นค่าตั้งต้นของ AI ยังไม่ผ่าน planning poker และการ์ดทุกใบสถานะ `draft` รอทีมตรวจรับ
PDF สร้างจากไฟล์ย่อยด้วยสคริปต์ เลขหน้าในสารบัญตรงทุกรายการ ไม่มีหน้าว่าง และดูภาพตัวอย่างหน้าสำคัญแล้ว	เกณฑ์วงเงินเป็นค่าสมมติ ไม่ได้เทียบกับระเบียบจริง และยังไม่ได้เปิด PDF ในโปรแกรมอ่านอื่นเทียบ

ตรวจแล้ว (มีหลักฐาน)	ยังไม่ได้ตรวจ
ความสอดคล้องของเนื้อหาสถาปัตยกรรมและ IA กับเอกสารอื่นถูกตรวจด้วยสคริปต์: ทุก API operation อยู่ในโมดูลเดียว ทุก NFR มีมาตรการ บทบาทของหน้าจอไม่เกินที่ API อนุญาต และทดสอบเชิงลบ 4 แบบจับได้ทุกแบบ	สถาปัตยกรรมและเทคโนโลยีที่เลือก (.NET, React, SQL Server, Redis) เป็นข้อเสนอที่ยังไม่ได้ทดลองสร้างต้นแบบ และ ADR ส่วนใหญ่ยังเป็นสถานะ `proposed`
wireframe ครบทุกหน้าจอ (10 จาก 10) ฟิลด์ ปุ่ม และข้อความผิดพลาดที่วาดตรงกับสเปกหน้าจอและกฎธุรกิจ ภาพและต้นฉบับมีจริง และแต่ละการ์ดงานอ้างถึง wireframe ของหน้าจอตนเอง	wireframe เป็นโครงความละเอียดต่ำแบบขาวดำ ไม่ใช่ visual design ไม่ได้ทดสอบกับผู้ใช้ และยังไม่ได้เปิดไฟล์ `.figcraft` ในแอป FigCraft แบบมีหน้าต่าง ตรวจจากภาพที่ render ด้วย CLI เท่านั้น

ตารางที่ 5-1 สิ่งที่ตรวจแล้วและสิ่งที่ยังไม่ได้ตรวจ